

Lab No 5:

Date:

## Collective Communications in MPI

### Objectives:

In this lab, student will be able to

1. Understand the usage of collective communication in MPI
2. Learn how to broadcast messages from root
3. Learn and use the APIs for distributing values from root and gathering the values in the root

### Collective Communication routines

When **all processes** in a group participate in a global communication operation, the resulting communication is called a **collective communication**.

#### MPI\_Bcast:

```
MPI_Bcast (Address, Count, Datatype, Root, Comm);
```

The process ranked **Root** sends the same message whose content is identified by the triple (Address, Count, Datatype) to all processes (including itself) in the communicator **Comm**.

#### MPI\_Scatter:

```
MPI_Scatter( SendBuff, Sendcount, SendDatatype, RecvBuff, Recvcount,  
RecvDatatype, Root, Comm);
```

Ensures that the **Root** process sends out personalized messages, which are in rank order in its send buffer, to all the N processes (including itself).

#### MPI\_Gather:

```
MPI_Gather( SendAddress, Sendcount, SendDatatype, RecvAddress, RecvCount,  
RecvDatatype, Root, Comm);
```

The **root** process receives a personalized message from all N processes. These N received messages are concatenated in rank order and stored in the receive buffer of the root process.

### Total Exchange:

In routine **MPI\_Alltoall()** each process sends a personalized message to every other process including itself. This operation is equivalent to N gathers, each by a different process and in all  $N^2$  messages are exchanged.

### Solved Example:

Implement a MPI program to read N values of the array in the root process. Distribute these N values among N processes. Every process finds the square of the value it received. Let every process return these value to the root and root process gathers and prints the result. Use collective communication routines.

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank, size, N, A[10], B[10], c, i;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank==0)
    {
        N=size;
        fprintf(stdout, "Enter %d values:\n", N);
        fflush(stdout);
        for(i=0; i<N; i++)
            scanf("%d", &A[i]);
    }
    MPI_Scatter(A, 1, MPI_INT, &c, 1, MPI_INT, 0, MPI_COMM_WORLD);
    fprintf(stdout, "I have received %d in process %d\n", c, rank);
    fflush(stdout);

    c=c*c;
    MPI_Gather(&c, 1, MPI_INT, B, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if(rank==0)
    {
        fprintf(stdout, "The Result gathered in the root \n");
        fflush(stdout);
        for(i=0; i<N; i++)
            fprintf(stdout, "%d \t", B[i]);
        fflush(stdout);
    }

    MPI_Finalize();
    return 0;
}
```

### Lab Exercises:

- 1) Implement a MPI program to read N values in the root process. Root process sends one value to each process. Every process receives it and finds the factorial of that number and returns it to the root process. Root process gathers the factorial and finds sum of it. Use N number of processes.
- 2) Implement a MPI program to read an integer value M and NXM elements into an 1D array in the root process, where N is the number of processes. Root process sends M elements to each process. Each process finds average of M elements it received and sends these average values to root. Root collects all the values and finds the total average. Use collective communication routines.
- 3) Implement a MPI program to read a string. Using N processes (string length is evenly divisible by N), find the number of non-vowels in the string. In the root process print number of non-vowels found by each process and print the total number of non-vowels.
- 4) Implement a MPI Program to read two strings S1 and S2 of same length in the root process. Using N processes including the root (string length is evenly divisible by N), produce the resultant string as shown below. Display the resultant string in the root process. Use Collective communication routines.

Example:

String S1: string      String S2: length      Resultant String : slternigtgh

- 5) Implement parallel selection sort [hint: Use merge sort after sorting subarrays using parallel selection sort]

### Additional Exercises:

- 1) Implement a MPI program to read a value M and NXM number of elements into 1D array in the root, where N is the total number of processes. Find the square of first M numbers, the cube of next M numbers and so on. Print the results in the root.
- 2) Implement a MPI program using collective communication functions, to replace all even elements of array A to 1 and replace all odd elements to 0 of size N. Display the resultant array A, count of all even and odd numbers in root process. Assume N is evenly divisible by number of processes.

Example :

Input Array (A):      1 2 3 4 5 6 7 8 9

Resultant Array (A): 0 1 0 1 0 1 0 1 0

Even (Count) = 4

Odd (Count) = 5